Chapter Two

Fundamentals of C++ Programming Language

A brief history of C++

- C++ was developed by Bjarne Stroustrup starting in 1979 at Bell Labs in Murray Hill, New Jersey, as an enhancement to the C language and originally named C with Classes but later it was renamed C++ in 1983
- C++ is regarded as a **middle-level** language, as it comprises a combination of both high-level and low-level language features
- **C++** runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX
- C++ is a statically typed, compiled, general-purpose, case-sensitive, free-form programming language that supports procedural, object-oriented, and generic programming

The structure of C++ program

- Lets look a simple example:

```
#include <iostream>
using namespace std;

// main() is where program execution begins.
int main() {
   cout << "Hello World"; // prints Hello World
   return 0;
}
```

- #➔ a signal to the preprocessor
- <iostream>➔ used by cin and cout to read and write information
- using namespace std;➔ we use all the things defined in std name space
- int main(){}➔ is special function, which will be called automatically at the start of a program and it is the main function where program execution begins.
- int is mentioned to state an integer return value
- **cout << "Hello World";** ➔ causes the message "Hello World" to be displayed on the screen
- Return 0; terminates main( )function and causes it to return the value 0 to the calling process

Compilation process of C++ program

- Open a text editor and add the code as above.
- Save the file as: hello.cpp
- Open the file on your code editor
- Compile and run the program, then
- You will be able to see ' Hello World ' printed on the window.
- semicolon is a statement terminator. That is, each individual statement must be ended with a semicolon. It indicates the end of one logical entity.
- For example, following are three different statements −

```
x = y;
y = y + 1;
add(x, y);
```

- A block is a set of logically connected statements that are surrounded by opening and closing braces. For example −

```
{
   cout << "Hello World"; // prints Hello World
   return 0;
}
```

- C++ does not recognize the end of the line as a terminator. For this reason, it does not matter where you put a statement in a line. For example −

```
x = y;
y = y + 1;
add(x, y);
```

is the same as

```
x = y; y = y + 1; add(x, y);
```

Input/output in C++

- **cin** is an instance of **istream** class. cin is said to be attached to the standard input device, which usually is the keyboard. The **cin** is used in conjunction with the stream extraction operator, which is written as >> Example:

```
#include <iostream>
using namespace std;
int main() {
  char name[50];
   cout << "Please enter your name: ";
  cin >> name;
  cout << "Your name is: " << name << endl;
}
```

- The stream extraction operator >> may be used more than once in a single statement. To request more than one datum you can use the following −
- cin >> name >> age;
- This will be equivalent to the following two statements −

  cin >> name;

  cin >> age;
- **cout** is an instance of **ostream** class. The cout object is said to be "connected to" the standard output device, which usually is the display screen. The **cout** is used in conjunction with the stream insertion operator, which is written as <<

Comments in C++

- A comment is a piece of descriptive text which explains some aspect of a program. Program comments are totally ignored by the compiler and are only intended for human readers. C++ provides two types of comment delimiters:
- //➔ single line comment
- /*

  */➔ multiple line comment
- Program comments are explanatory statements that you can include in the C++ code. These comments help anyone reading the source code. All programming languages allow for some form of comments.

C++ tokens (Keywords, Identifiers, Constants, and Operators)

Keywords

- Reserved/Key words have a unique meaning within a C++ program. These symbols must not be used for any other purposes. All reserved words are in lower-case letters. Example:

Asm	auto	bool	break	case	Catch
const_cast	class	const	char	continue	default
dynamic_cast	do	double	delete	else	Enum
Explicit	extern	false	float	for	Friend
Goto	if	inline	Int	long	mutable
Namespace	new	operator	private	protected	public
reinterpret_cast	register	return	short	signed	Sizeof
static_cast	static	struct	switch	template	This
Throw	true	try	typedef	typeid	typename
Union	unsigned	using	virtual	void	volatile
wchar_t					

Identifiers

- An identifier is name associated with a function or data object and used to refer to that function or data object. An identifier must:
  - Start with a letter or underscore
  - Consist only of letters, the digits 0-9, or the underscore symbol _
  - Not be a reserved word
- C++ is case-sensitive. That is lower-case letters are treated as distinct from upper-case letters. Thus the word NUM different from the word num or the word Num. Identifiers can be used to identify variable or constants or functions. Function identifier is an identifier that is used to name a function.

Constants

- Constants refer to fixed values that the program may not alter and they are called **literals**.
- Constants can be of any of the basic data types and can be divided into Integer Numerals, Floating-Point Numerals, Characters, Strings and Boolean Values.
- There are two simple ways in C++ to define constants −
  - Using **#define** preprocessor.
  - Using **const** keyword.
- it is a good programming practice to define constants in CAPITALS.

Operators

- An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. C++ is rich in built-in operators and provide the following types of operators −
  - Arithmetic Operators
  - Relational Operators
  - Logical Operators
  - Bitwise Operators
  - Assignment Operators
  - Misc Operators

Arithmetic Operators

- Assume variable A holds 10 and variable B holds 20, then

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiplies both operands	A * B will give 200
/	Divides numerator by de-numerator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	Increment operator, increases integer value by one	A++ will give 11
--	Decrement operator, decreases integer value by one	A-- will give 9

Relational Operators

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Logical Operators

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then condition becomes true.	(A && B) is false.
||	Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true.	(A || B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true, then Logical NOT operator will make false.	!(A && B) is true.

Bitwise Operators

p	q	p & q	p | q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume if A = 60; and B = 13; now in binary format they will be as follows −
A = 0011 1100
B = 0000 1101

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12 which is 0000 1100
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B) will give 61 which is 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49 which is 0011 0001
~	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61 which is 1100 0011 in 2's complement form due to a signed binary number.
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 0000 1111

Assignment Operators

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand.	C = A + B will assign value of A + B into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	Bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2
|=	Bitwise inclusive OR and assignment operator.	C |= 2 is same as C = C | 2

Misc Operators

Sr.No	Operator & Description
1	**sizeof** sizeof operator returns the size of a variable. For example, sizeof(a), where 'a' is integer, and will return 4.
2	**Condition ? X : Y** Conditional operator (?). If Condition is true then it returns value of X otherwise returns value of Y.
3	**,** Comma operator causes a sequence of operations to be performed. The value of the entire comma expression is the value of the last expression of the comma-separated list.
4	**. (dot) and -> (arrow)** Member operators are used to reference individual members of classes, structures, and unions.
5	**Cast** Casting operators convert one data type to another. For example, int(2.2000) would return 2.
6	**&** Pointer operator & returns the address of a variable. For example &a; will give actual address of the variable.
7	* Pointer operator * is pointer to a variable. For example *var; will pointer to a variable var.

Operators Precedence in C++

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

What is a variable?

- A variable is a symbolic name for a memory location in which data can be stored and subsequently recalled. Variables are used for holding data values so that they can be utilized in various computations in a program. All variables have two important attributes:
  - A type, which is, established when the variable is defined (e.g., integer, float, character). Once defined, the type of a C++ variable cannot be changed.
  - A value, which can be changed by assigning a new value to the variable. The kind of values a variable can assume depends on its type. For example, an integer variable can only take integer values (e.g., 2, 100, -12) not real numbers like 0.123.

Variable Declaration

- Declaring a variable means defining (creating) a variable. You create or define a variable by stating its type, followed by one or more spaces, followed by the variable name and a semicolon.
- The variable name can be any combination of letters, but cannot contain spaces and the first character must be a letter or an underscore.
- Variable names cannot also be the same as keywords used by C++.
- Good variable names tell you what the variables are for; using good names makes it easier to understand the flow of your program. The following statement defines an integer variable called myAge:

  int myAge;

- **IMPORTANT**- Variables must be declared before used!

Creating More Than One Variable at a Time

- You can create more than one variable of the same type in one statement by writing the type and then the variable names, separated by commas. For example:

  int myAge, myWeight; // two int variables

  long area, width, length; // three longs

- As you can see, myAge and myWeight are each declared as integer variables. The second line declares three individual long variables named area, width, and length. However keep in mind that you cannot mix types in one definition statement.

Assigning Values to Your Variables

- You assign a value to a variable by using the assignment operator (=). Thus, you would assign 5 to Width by writing

```
int  Width;
Width = 5;
```

- You can combine these steps and initialize Width when you define it by writing

```
int Width = 5;
```

- Just as you can define more than one variable at a time, you can initialize more than one variable at creation. For example:

```
// create two int variables and initialize them
int width = 5, length = 7;
```

Basic Data Types

Type	Keyword
Boolean	bool
Character	char
Integer	int
Floating point	float
Double floating point	double
Valueless	void
Wide character	wchar_t

- Several of the basic types can be modified using one or more of these type modifiers −
  - signed
  - unsigned
  - short
  - long
- We can using **sizeof()** operator to get size of various data types
- You can create a new name for an existing type using **typedef**

  typedef type newname;

  typedef int feet;

  feet distance;

Enumerated Types

- An enumerated type declares an optional type name and a set of zero or more identifiers that can be used as values of the type. Each enumerator is a constant whose type is the enumeration.
- Creating an enumeration requires the use of the keyword **enum**. The general form of an enumeration type is −

  enum enum-name { list of names } var-list;

  enum color { red, green, blue } c;

  c = blue;
- By default, the value of the first name is 0, the second name has the value 1, and the third has the value 2, and so on
- But you can give a name, a specific value by adding an initializer

Scope of variables

- there are three places, where variables can be declared −
  - Inside a function or a block which is called local variables,
  - In the definition of function parameters which is called formal parameters.
  - Outside of all functions which is called global variables.

Local Variables

- Variables that are declared inside a function or block are local variables. They can be used only by statements that are inside that function or block of code.

Example:

Global Variables

- Global variables are defined outside of all the functions, usually on top of the program. The global variables will hold their value throughout the life-time of your program.
- A global variable can be accessed by any function. That is, a global variable is available for use throughout your entire program after its declaration.

Initializing Local and Global Variables

- When a local variable is defined, it is not initialized by the system, you must initialize it yourself. Global variables are initialized automatically by the system